REDYNOX INTERNSHIP REPORT – TASK 2: WEB APPLICATION SECURITY

1. Introduction

This report contains the completion of Task 2: *Web Application Security*. The objective of this task was to understand common web application vulnerabilities and how attackers exploit them using practical tools such as WebGoat and OWASP ZAP.

The task involved setting up a vulnerable web application, performing automated and manual security testing, identifying vulnerabilities, and suggesting mitigation strategies.

2. Task Overview

Web Application Security

2.1 Objective

The objective of this task was to:

- Understand common web vulnerabilities
- Perform vulnerability scanning
- Manually exploit security flaws
- Learn mitigation techniques

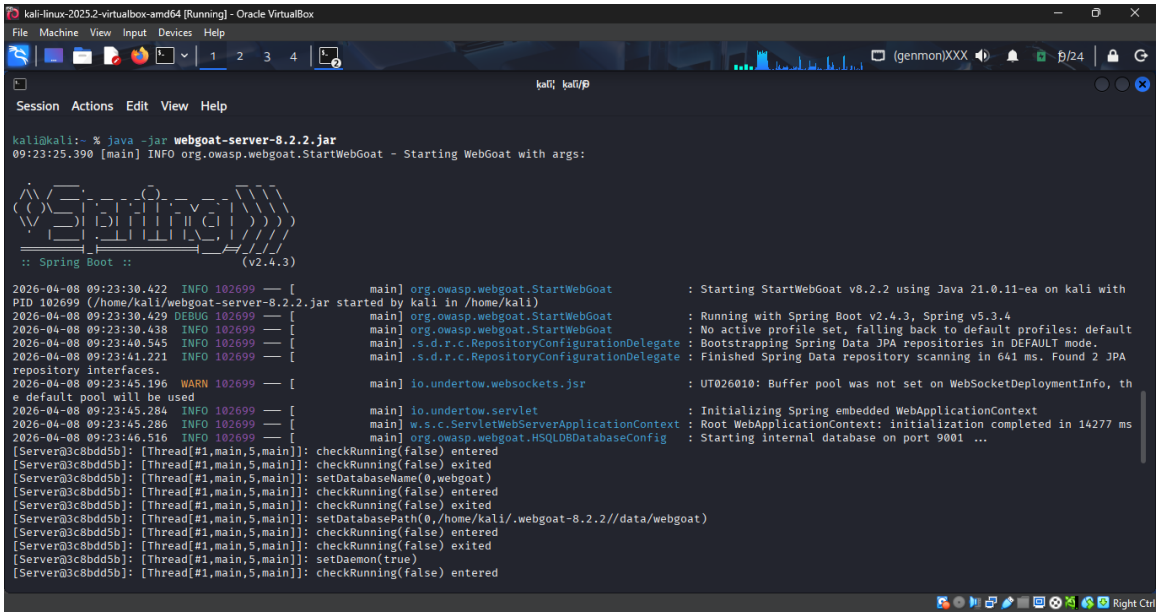
Tools and Methods Used

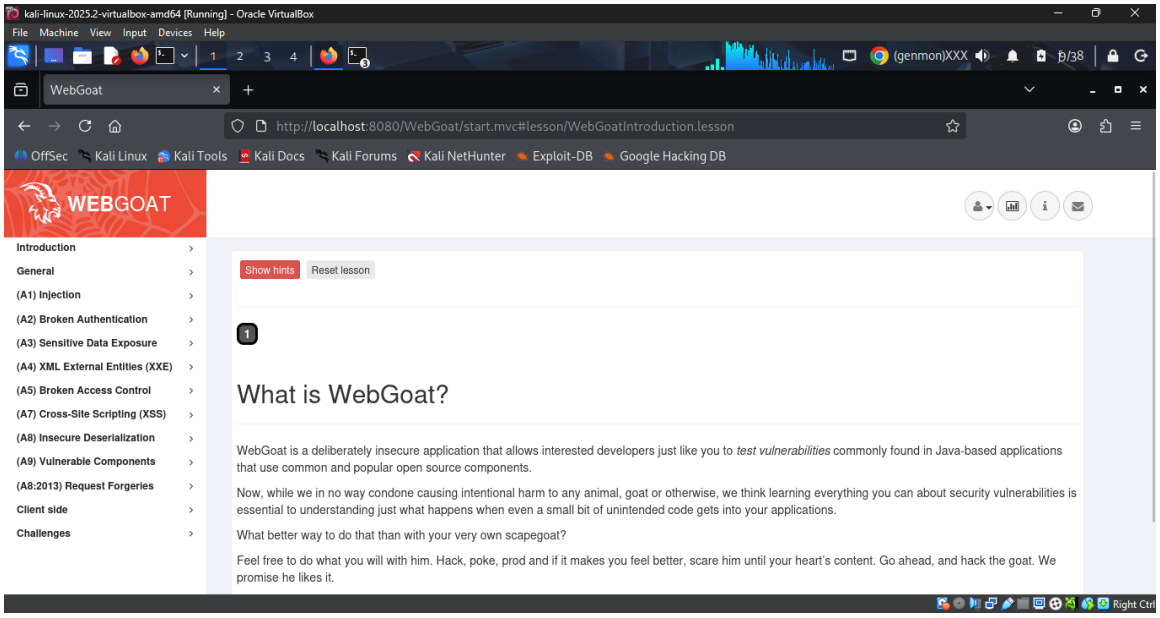
- WebGoat – for practicing vulnerabilities
- OWASP ZAP – for automated scanning and analysis
- Kali Linux – testing environment

Steps Taken

1. WebGoat was installed and launched locally

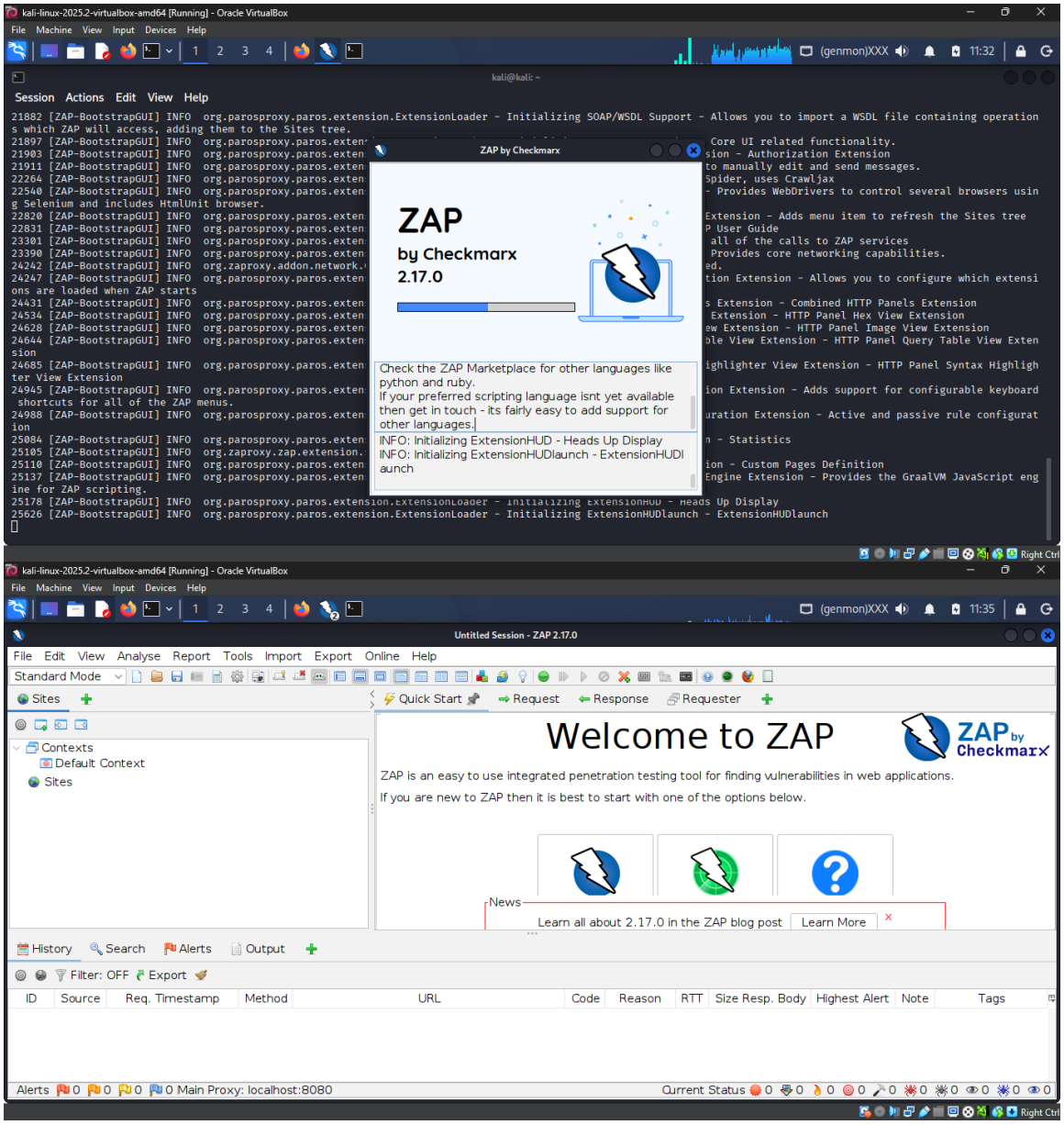
The application was created, and the login was successful (http://localhost:8080/WebGoat)





2. Configure OWASP ZAP

ZAP was launched and configured
A port conflict on 8080 was resolved by changing ZAP proxy to port 8090
Manual Explore and Automated Scan features were used



3. Capturing traffic via ZAP browser

WebGoat was accessed through the ZAP browser
User interactions (SQL question) was performed
HTTP requests were captured in the Sites panel

kali-linux-2025.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

(genmon)XXX 13:04

Untitled Session - 20260408-123846 - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites

- https://fonts.googleapis.com
- https://waitlist.propertyloop.ng
- https://propertyloop.ng
- http://propertyloop.ng
- http://localhost:8080
  - GET:WebGoat
  - WebGoat

Quick Start | Request | Response | Requester

URL to explore: http://localhost:8080/WebGoat Select...

Enable HUD: ☑

Explore your application: Launch Browser Firefox

You can also use browsers that you don't launch from ZAP, but will need to configure them to proxy through ZAP and to import the ZAP root CA certificate.

History | Search | Alerts | Output | WebSockets | Active Scan

Filter: OFF Export

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
272	Pro...	4/8/26, 12:48:44 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	18...	7,362 bytes			
273	Pro...	4/8/26, 12:48:50 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	13...	7,362 bytes			JSON
274	Pro...	4/8/26, 12:48:49 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	14...	1,396 bytes			
275	Pro...	4/8/26, 12:48:55 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	12...	1,396 bytes			JSON
276	Pro...	4/8/26, 12:48:55 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	20...	7,362 bytes			JSON
277	Pro...	4/8/26, 12:49:00 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	16...	1,396 bytes			JSON
278	Pro...	4/8/26, 12:49:00 PM	GET	http://localhost:8080/WebGoat/service/lesso...	200	OK	17...	7,362 bytes			
279	Pro...	4/8/26, 12:48:18 PM	GET	https://waitlist.propertyloop.ng/img/127983...	206	Partial Co...	1....	12,047,806 b...			Comment

Alerts 2 6 6 7 Main Proxy: localhost:8090 Current Status 0 0 2 0 0 0 0 0 0 0 0

Right Ctrl

kali-linux-2025.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

(genmon)XXX 12:50

Untitled Session - 20260408-123846 - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites

- Contexts
  - Default Context
- Sites
  - https://fonts.gstatic.com
  - https://fonts.googleapis.com
  - https://waitlist.propertyloop.ng
  - https://propertyloop.ng
  - http://propertyloop.ng
  - http://localhost:8080

Quick Start | Request | Response | Requester

Manual Explore

ZAP by Checkmarx

This screen allows you to launch the browser of your choice so that you can explore your application while proxying through ZAP.

The ZAP Heads Up Display (HUD) brings all of the essential ZAP functionality into your browser.

URL to explore: http://localhost:8080/WebGoat Select...

Enable HUD: ☑

Explore your application: Launch Browser Firefox

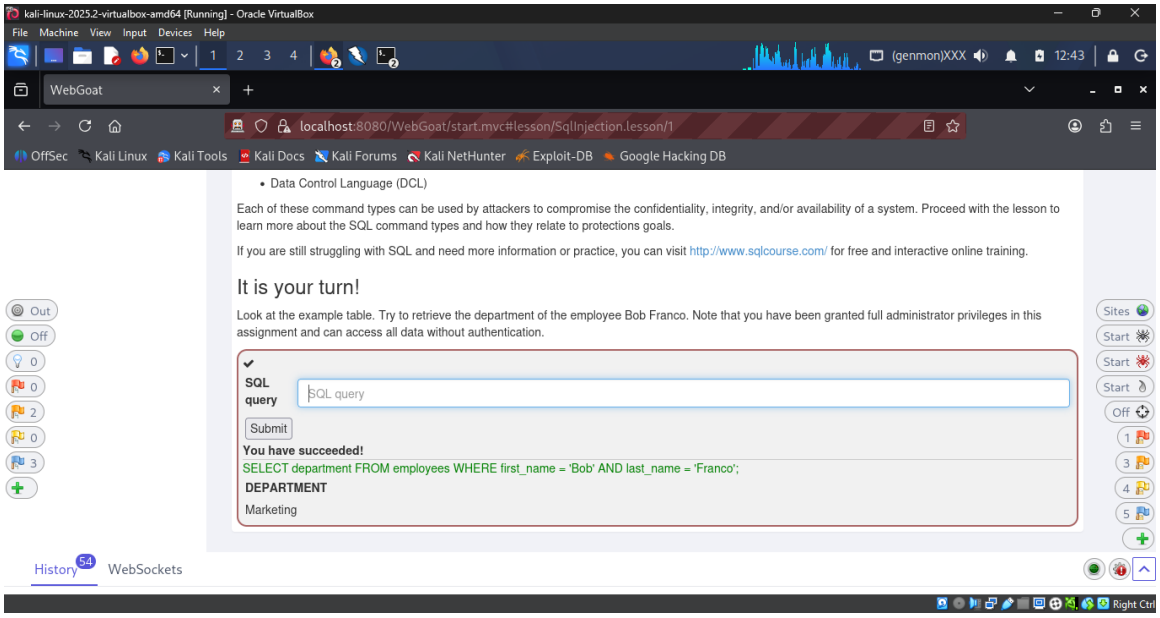
History | Search | Alerts | Output | WebSockets

Channel: -- All Channels -- Filter:OFF

Channel	↔	Timestamp	Opcode	Bytes	Payload
#2.57	⇐	08/04/2026, 12:48:55.464	1=TEXT	314	{"event.type":"href.added","e...
#1.374	⇐	08/04/2026, 12:48:55.529	1=TEXT	310	{"event.type":"href.added","e...
#2.58	⇐	08/04/2026, 12:48:55.541	1=TEXT	310	{"event.type":"href.added","e...
#1.375	⇒	08/04/2026, 12:48:56.141	1=TEXT	64	{"component":"hud","type":"vi...
#2.59	⇒	08/04/2026, 12:48:59.12	1=TEXT	64	{"component":"hud","type":"vi...

Alerts 1 6 6 7 Main Proxy: localhost:8090 Current Status 0 0 0 0 0 0 0 0 0 0 0

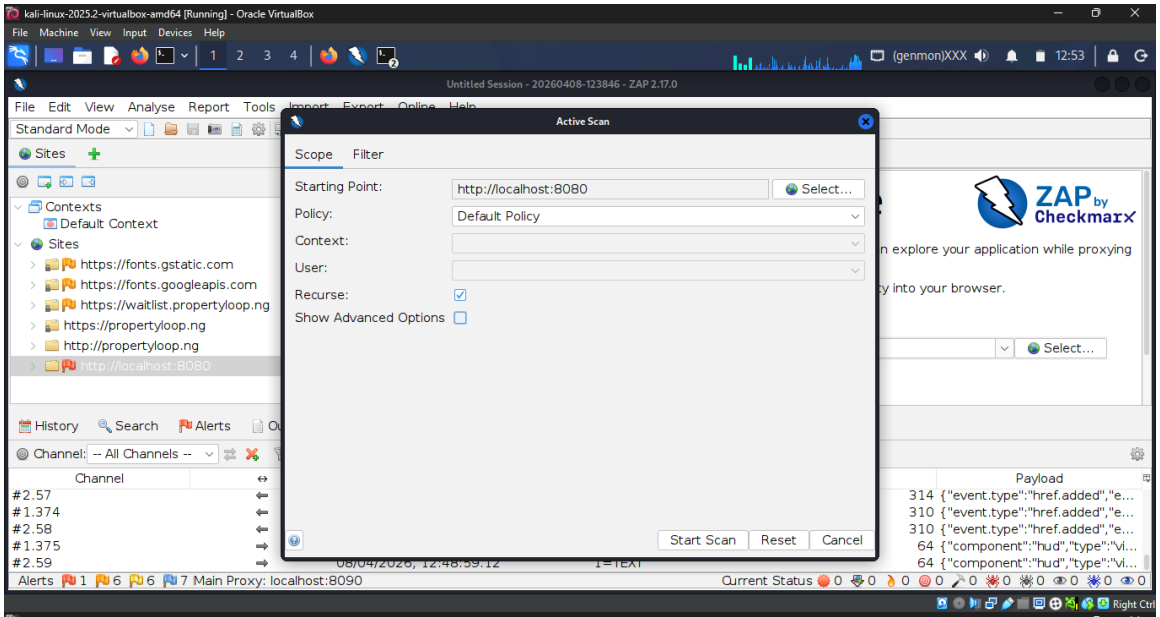
Right Ctrl

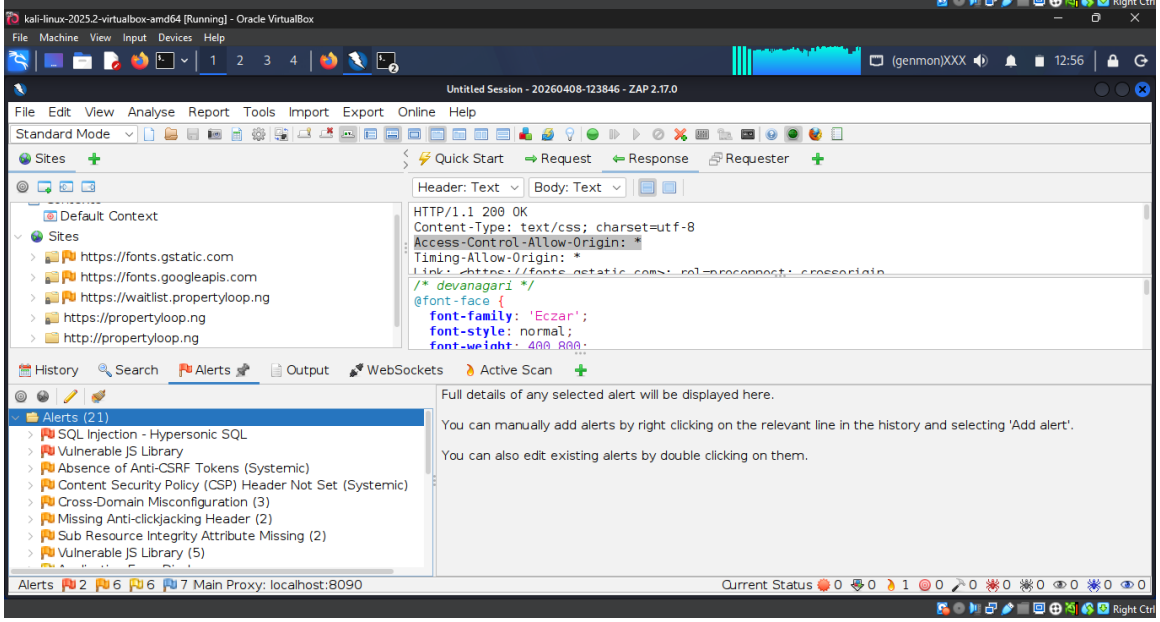


4. Manual Vulnerability Scanning

An Active Scan was performed on the WebGoat application

ZAP generated alerts based on detected vulnerabilities



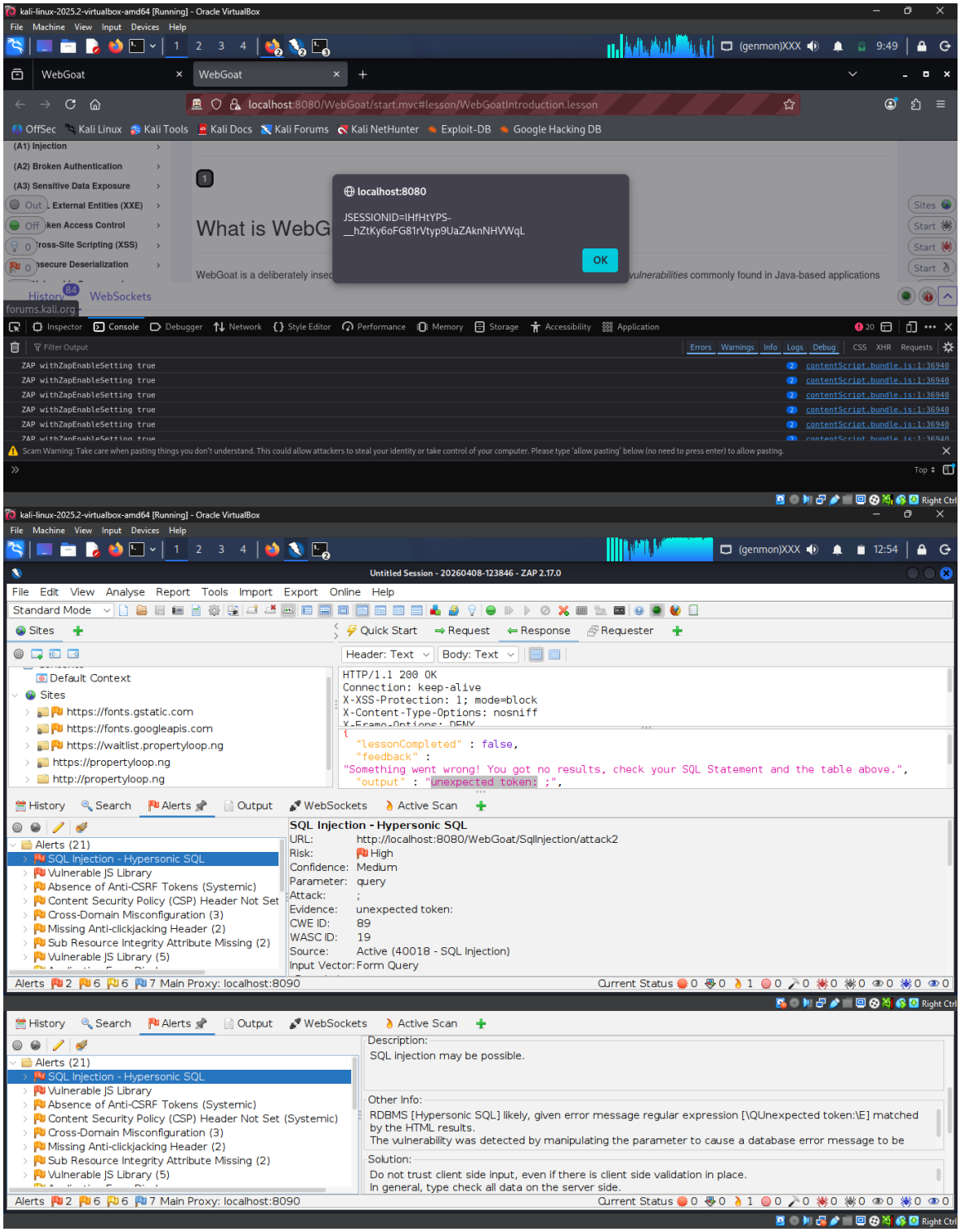


Vulnerability Analysis and Exploitation

XSS:

XSS was tested using:

<script>alert('XSS')</script>



Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages, which are then executed in the victim's browser.

Additional Observation (Cookies):

- Using alert(document.cookie) showed that cookies are shared across tabs of the same domain
- This demonstrates how attackers can steal session cookies

CSRF:

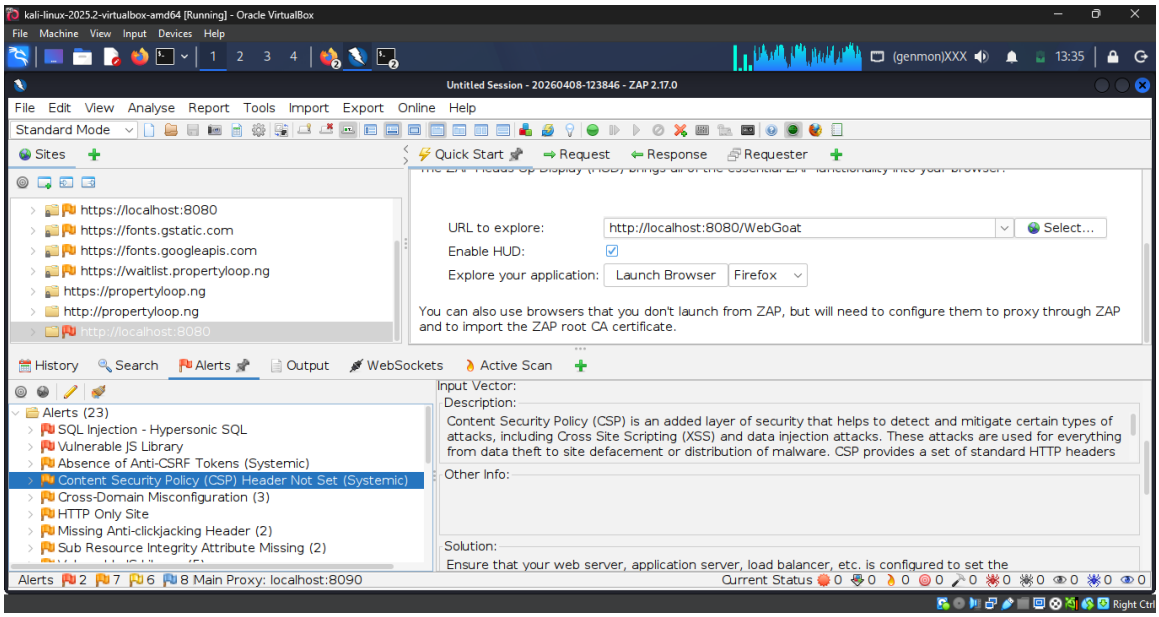
CSRF vulnerabilities were identified through ZAP alerts

Forms lacking proper request validation were observed

CSRF attacks trick users into performing unintended actions on a web application without their consent.

Mitigation:

- Use CSRF tokens
- Validate request origin



Challenges Faced and How They Were Overcome

- **Port conflict (8080 already in use)**
  → Resolved by reconfiguring ports and restarting services

- **ZAP Sites panel not showing data**
  → Fixed by using Automated Scan and proper browser interaction
- **WebGoat server errors (StackOverflow)**
  → Resolved by restarting the application

Discussion: How Security Measures Protect Web Applications

The vulnerabilities identified highlight the importance of secure coding practices. Implementing input validation, parameterized queries, secure authentication mechanisms, and proper request validation can significantly reduce the risk of attacks.

Security tools like OWASP ZAP help identify weaknesses early, allowing developers to fix them before exploitation.

Conclusion

This task provided practical exposure to web application security testing. It enhanced understanding of how attackers exploit vulnerabilities and how defensive mechanisms can be implemented.

The experience gained from using WebGoat and OWASP ZAP is essential for real-world cybersecurity applications.

Closing Remarks

The task was highly educational and provided hands-on experience in ethical hacking techniques. It improved technical skills in vulnerability analysis and strengthened knowledge of secure web practices.

Future work can include testing more complex applications and exploring advanced security tools.